Campus Event Management System

Context

A university organizes campus events — technical workshops, hackathons, placement sessions, club activities, cultural programmes, and student competitions — that vary in duration, expected participation, venue requirements and organizational needs.

Currently, events are coordinated through messages, spreadsheets and informal communication. As volume grows, the university faces conflicting venue requests, venues selected without considering suitability or capacity, limited visibility into request status, delays in reviewing and confirming events, venues remaining unavailable after cancellations, temporary unavailability due to maintenance or renovation, and different stakeholders requiring different information and capabilities.

Design and build a working prototype of a Campus Event Management System. Your team must study the business situation, identify the relevant stakeholders, and determine the workflows, business rules and validations required for the system to operate reliably. The solution should support a meaningful campus event lifecycle, from an initial request to an appropriate outcome.

Multiple valid approaches are possible. Your solution should be internally consistent, usable and explainable by the team.

Business Conditions

While designing your solution, consider the following minimum business conditions:

1. An event is associated with a defined start date-time, end date-time and venue.
2. A venue must not be committed to overlapping active event requests or confirmed events.
3. A conflicting request must be prevented before it is submitted.
4. A venue may be unavailable because of maintenance, renovation or another operational restriction.
5. A successfully submitted event request must pass through a review process before it becomes a confirmed event.
6. A rejected request is considered closed. Rejecting an event request must release the associated venue commitment so that the venue becomes available for another valid request during that period. To submit revised event information, the requester must create a new request.
7. An appropriate end user must be able to cancel a confirmed event.
8. Cancelling an event must release the associated venue commitment.
9. Partial bookings, split bookings and automatic alternative venue suggestions are not required.
10. If the selected venue cannot satisfy the request, the request must not proceed, and the system should communicate the reason clearly.

Your team may introduce additional assumptions, provided they are documented and applied consistently.

Initial Campus Data

Use the following as initial business data. Teams may add reasonable details or additional sample records for testing and demonstration.

Venue	Type	Capacity	Current Information
Seminar Hall A	Seminar Hall	120	Confirmed event on 4 Aug 2026, 10:00–12:00
Main Auditorium	Auditorium	500	Available
Conference Room C	Conference Room	40	Blocked for renovation on 4 Aug 2026, 09:00–18:00
Central Amphitheatre	Open Venue	300	Available
Innovation Lab	Computer Lab	60	Confirmed event on 6 Aug 2026,14:00–17:00

Persistence can be in-memory, local files, JSON, CSV, SQLite or another lightweight local approach.

Constraints

- 90 minutes to complete.
- Build a locally runnable prototype with a UI (Streamlit, web-based, or any framework).
- Use Python as the preferred language. C++ or Java are also acceptable.
- No cloud deployment, external APIs, paid services, complex infrastructure, or access to university production systems.
- Authentication is optional. If implemented, explain why it was prioritized.
- Each team must use a shared GitHub repository reflecting meaningful and incremental collaboration. Commit count alone will not measure individual contribution; evaluators may use repository history as supporting evidence of how the team divided, integrated and validated its work.

Deliverables

1. **Working Prototype**: A locally runnable application demonstrating a meaningful event management lifecycle with connected workflows, consistent handling of event and venue states, relevant business validations, and conflict/failure handling. At least one working end-to-end flow must be demonstrated. A smaller, reliable and well-explained solution may score higher than a larger application containing generated code the team cannot validate or explain.
2. **GitHub Repository**: Must contain source code, run instructions (prerequisites, startup, setup), documented assumptions, scope prioritized, features excluded, important trade-offs, known limitations, and the consulting pitch.
3. **Two-Slide Consulting Pitch:**
    a. Slide 1 — Problem Understanding: Your understanding of the problem, stakeholders/personas identified, workflows prioritized, important business rules discovered, key assumptions, and scope selected for 90 minutes.
    b. Slide 2 — Solution & Engineering Decisions: High-level approach, important design decisions, key trade-offs, testing/validation approach, known limitations, and what the team would improve next.
4. **Final Demonstration Covering**: business problem understanding, stakeholders and workflows identified, the working application, a successful end-to-end scenario, a conflict/invalid/failure scenario, design decisions, testing performed, assumptions and trade-offs, known limitations, and recommended next steps. Evaluators may provide an additional scenario and may direct questions to any team member — each member should be prepared to explain their contribution, one business rule, one part of the solution, one assumption or trade-off, and how their work was validated.

Use of AI Tools

This is an open-book, AI-assisted software engineering challenge. Teams may use GitHub Copilot, Copilot Chat, Claude, GPT-based models, IDE assistance, official documentation, internet search, and AI-generated code, tests, diagrams, documentation and presentation content.

Using AI is not considered cheating. However, the team remains responsible for understanding the problem, identifying requirements, choosing the solution design, reviewing generated output, detecting incorrect or unnecessary suggestions, running and validating generated code, ensuring the implementation matches the business problem, and explaining every submitted artefact.

The objective is not to demonstrate how quickly AI can generate an application. The objective is to demonstrate how effectively the team can direct, question, validate and improve AI-assisted work while retaining ownership of the final solution.

Evaluation Criteria

Criterion	Weight	What We Look For
Problem Understanding	20%	Did the team identify stakeholders, workflows, business rules and scope from the problem statement?

Implementation & Completeness	20%	Does the solution work? How many flows are functional? Is the code well-structured and explainable?
AI Usage	20%	How effectively did the team leverage AI for design, code, iteration and validation?
Collaboration & Communication	20%	How well did the team collaborate? Is the consulting pitch clear and reasoned?
Quiz	20%	Hackathon quiz conducted earlier

Bonus Feature

If your core system is functional and the primary use cases are working, implement one additional capability that enhances the system. Think about what would make this tool genuinely more useful for a campus community — notifications, analytics, calendar integration, smart suggestions, or something else entirely. Creativity counts.

> Note: Bonus is only considered if the core functionality is demonstrably working.

Duration: 90 minutes
Team Size: As announced
Tools: VS Code + GitHub Copilot